# EE323 Digital Signal Processing
# Lab 7 - Digital Filter Design

## 7.1 Introduction

This laboratory covers some basic examples of FIR and IIR filters, and then introduces the concepts of FIR filter design. Systematic methods of designing both FIR and IIR filters are followed.

## 7.2 Background on Digital Filters

In digital signal processing applications, it is often necessary to change the relative amplitudes of frequency components or remove undesired frequencies of a signal. This process is called **filtering**. Digital filters are used in a variety of applications. Digital filters are used in audio systems in the previous laboratories that allow the listener to adjust the bass (low-frequency energy) and the treble (high frequency energy) of audio signals.

Digital filter design requires the use of both frequency domain and time domain techniques. This is because filter design specifications are often given in the frequency domain, but filters are usually implemented in the time domain with a difference equation. Typically, frequency domain analysis is done using the z-transform and the discrete-time Fourier Transform (DTFT).

In general, a linear and time-invariant causal digital filter with input $x[n]$ and output $y[n]$ may be specified by its difference equation

$$y[n] = \sum_{i=0}^{N-1} b_i\, x[n-i] - \sum_{k=1}^{M-1} a_k\, y[n-k] \tag{7.1}$$

where $b_i$ and $a_k$ are coefficients which parameterize the filter. This filter is said to have $N-1$ zeros and $M-1$ poles. Each new value of the output signal, $y[n]$, is determined by past values of the output, and by present and past values of the input. The impulse response, $h[n]$, is the response of the filter to an input of $\delta[n]$, and is therefore the solution to the recursive difference equation

$$h[n] = \sum_{i=0}^{N-1} b_i\, \delta[n-i] - \sum_{k=1}^{M-1} a_k\, h[n-k] \tag{7.2}$$

There are two general classes of digital filters: infinite impulse response (IIR) and finite impulse response (FIR). The FIR case occurs when $a_k = 0$, for all $k$. Such a filter is said to have no poles, only zeros. In this case, the difference equation in (7.2) becomes

$$h[n] = \sum_{i=0}^{N-1} b_i\, \delta[n-i] \tag{7.3}$$

Since (7.3) is no longer recursive, the impulse response has finite duration $N$.

In the case where $a_k \neq 0$, the difference equation usually represents an IIR filter. In this case,

(7.2) will usually generate an impulse response which has non-zero values as $n \to \infty$.

The $z$-transform is the major tool used for analyzing the frequency response of filters and their difference equations. The $z$-transform of a discrete-time signal, $x[n]$, is given by

$$X(z) = \sum_{n=-\infty}^{\infty} x[n]\, z^{-n} \tag{7.4}$$

where $z$ is a complex variable. The DTFT may be thought of as a special case of the $z$-transform where $z$ is evaluated on the unit circle in the complex plane.

$$\begin{aligned} X\left(e^{j\omega}\right) &= X(z)|_{z=e^{j\omega}} \\ &= \sum_{n=-\infty}^{\infty} x[n] e^{-j\omega n} \end{aligned} \tag{7.5}$$

From the definition of the z-transform, a change of variable $m = n - K$ shows that a delay of $K$ samples in the time domain is equivalent to multiplication by $z^{-K}$ in the $z$-transform domain.

$$\begin{aligned} x[n-K] \xleftrightarrow{Z} \sum_{n=-\infty}^{\infty} x[n-K] z^{-n} &= \sum_{m=-\infty}^{\infty} x[m] z^{-(m+K)} \\ &= z^{-K} \sum_{m=-\infty}^{\infty} x[m] z^{-m} = z^{-K} X(z) \end{aligned} \tag{7.6}$$

We may use this fact to re-write (7.1) in the z-transform domain, by taking $z$-transforms of both sides of the equation:

$$\begin{aligned} Y(z) &= \sum_{i=0}^{N-1} b_i\, z^{-i} X(z) - \sum_{k=1}^{M-1} a_k\, z^{-k} Y(z) \\ Y(z)\left(1 + \sum_{k=1}^{M-1} a_k z^{-k}\right) &= X(z) \sum_{i=0}^{N-1} b_i\, z^{-i} \\ H(z) \triangleq \frac{Y(z)}{X(z)} &= \frac{\sum_{i=0}^{N-1} b_i z^{-i}}{1 + \sum_{k=1}^{M-1} a_k z^{-k}} \end{aligned} \tag{7.7}$$

From this formula, we see that any filter which can be represented by a linear difference equation with constant coefficients has a rational transfer function (i.e. a transfer function which is a ratio of polynomials). From this result, we may compute the frequency response of the filter by evaluating $H(z)$ on the unit circle:

$$H\left(e^{j\omega}\right) = \frac{\sum_{i=0}^{N-1} b_i e^{-j\omega i}}{1 + \sum_{k=1}^{M-1} a_k e^{-j\omega k}} \tag{7.8}$$

There are many different methods for implementing a general recursive difference equation of the form (7.1). Depending on the application, some methods may be more robust to quantization error, require fewer multiplies or adds, or require less memory. Figure 7.1 shows a system diagram known as the direct form implementation; it works for any discrete-time filter described by the difference equation in (7.1). Note that the boxes containing the symbol $z^{-1}$ represent unit delays, while a

parameter written next to a signal path represents multiplication by that parameter.

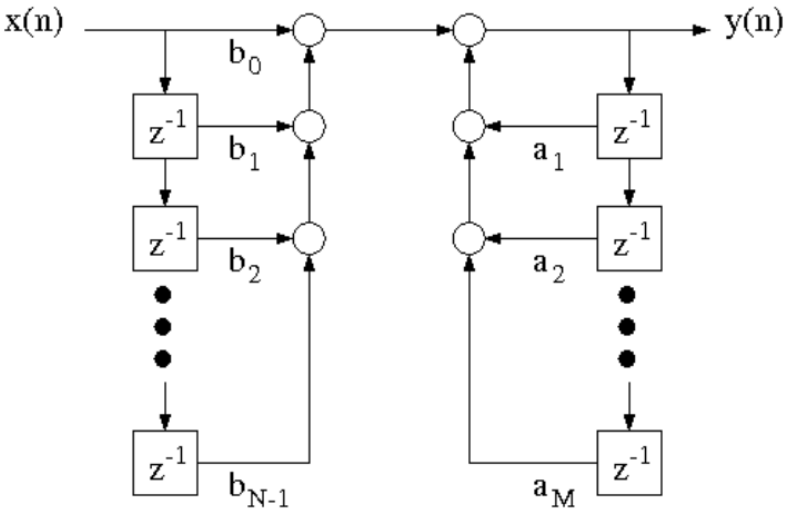


Figure 7.1: Direct form implementation for a discrete-time filter described by a linear recursive difference equation.

# 7.3 Design of a Simple FIR Filter

Download the files, **nspeech1.mat** and **DTFT.m** for the following section.

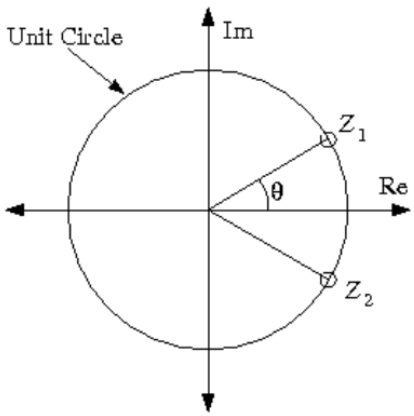


Figure 7.2: Location of two zeros for simple a FIR filter.

To illustrate the use of zeros in filter design, you will design a simple second order FIR filter with the two zeros on the unit circle as shown in Figure 7.2. In order for the filter's impulse response to be real-valued, the two zeros must be complex conjugates of one another:

$$z_1 = e^{j\theta}, \text{ and } z_2 = e^{-j\theta}$$

where $\theta$ is the angle of $z_1$ relative to the positive real axis. We will see later that $\theta \epsilon [0, \pi]$ may be interpreted as the location of the zeros in the frequency response.

The transfer function for this filter is given by

$$H_f(z) = (1 - z_1 z^{-1})(1 - z_2 z^{-1}) = (1 - e^{j\theta} z^{-1})(1 - e^{-j\theta} z^{-1}) = 1 - 2\cos\theta z^{-1} + z^{-2} \tag{7.9}$$

Use this transfer function to determine the difference equation for this filter. Then draw the corresponding system diagram and compute the filter's impulse response $h[n]$.

This filter is an FIR filter because it has impulse response $h[n]$ of finite duration. Any filter with only zeros and no poles other than those at 0 and $\pm\infty$ is an FIR filter. Zeros in the transfer function represent frequencies that are not passed through the filter. This can be useful for removing unwanted frequencies in a signal. The fact that $H_f(z)$ has zeros at $e^{\pm j\theta}$ implies that $H_f\left(e^{\pm j\theta}\right) = 0$. This means that the filter will not pass pure sine waves at a frequency of $\omega = \theta$.

Use Matlab to compute and plot the magnitude of the filter's frequency response $\left|H_f(e^{j\omega})\right|$ as a function of $\omega$ on the interval $-\pi < \omega < \pi$, for the following three values of $\theta$:

i) $\theta = \pi/6$, ii) $\theta = \pi/3$ iii) $\theta = \pi/2$

**INLAB REPORT:** Submit the difference equation, system diagram, and the analytical expression of the impulse response for the filter $H_f(z)$. Also submit the plot of the magnitude response for the three values of $\theta$. Explain how the value of $\theta$ affects the magnitude of the filter's frequency response.

In the next experiment, we will use the filter $H_f(z)$ to remove an undesirable sinusoidal interference from a speech signal. To run the experiment, first download the audio signal **nspeech1.mat**, and the M-file **DTFT.m.** Load **nspeech1.mat** into Matlab using the command **load nspeech1**. This will load the signal **nspeech1** into the workspace. Play **nspeech1** using the **sound** command, and then **plot** 101 samples of the signal for the time indices (100:200).

We will next use the **DTFT** command to compute samples of the DTFT of the audio signal. The DTFT command has the syntax

[X ,w]=DTFT(x ,M)

where x is a signal which is assumed to start at time $n = 0$, and $M$ specifies the number of output points of the DTFT. The command [X ,w]=DTFT(x,0) will generate a DTFT that is the same duration as the input; if this is not sufficient, it may be increased by specifying $M$. The outputs of the function are a vector X containing the samples of the DTFT, and a vector w containing the corresponding frequencies of these samples.

Compute the magnitude of the DTFT of 1001 samples of the audio signal for the time indices (100:1100). Plot the magnitude of the DTFT samples versus frequency for $|\omega| < \pi$. Notice that there are two large peaks corresponding to the sinusoidal interference signal. Use the Matlab **max** command to determine the exact frequency of the peaks. This will be the value of $\theta$ that we will use for filtering with $H_f(z)$.

**HINT:** Use the command [Xmax , Imax]=**max**(**abs**(X)) to find the value and index of the maximum element in X . $\theta$ can be derived using this index.

Write a Matlab function **FIRfilter**(x) that implements the filter $H_f(z)$ with the measured value of

$\theta$ and outputs the filtered signal (**HINT:** Use convolution). Apply the new function **FIRfilter** to the **nspeech1** vector to attenuate the sinusoidal interference. Listen to the filtered signal to hear the effects of the filter. Plot 101 samples of the signal for the time indices (100:200), and plot the magnitude of the DTFT of 1001 samples of the filtered signal for the time indices (100:1100).

**INLAB REPORT:** For both the original audio signal and the filtered output, hand in the following:

- The time domain plot of 101 samples.
- The plot of the magnitude of the DTFT for 1001 samples.

Also hand in the code for the **FIRfilter** filtering function. Comment on how the frequency content of the signal changed after filtering. Is the filter we used a lowpass, highpass, bandpass, or a bandstop filter? Comment on how the filtering changed the quality of the audio signal.

# 7.4 Design of a Simple IIR Filter

Download the file **pcm.mat** for the following section.

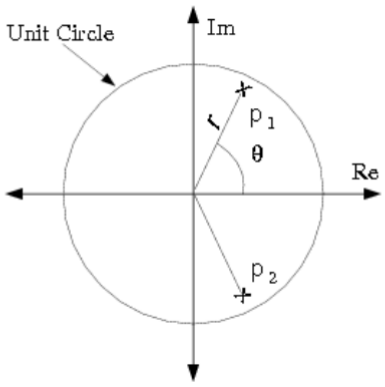


**Figure 7.3**: Location of two poles for a simple IIR filter.

While zeros attenuate a filtered signal, poles amplify signals that are near their frequency. In this section, we will illustrate how poles can be used to separate a narrow-band signal from adjacent noise. Such filters are commonly used to separate modulated signals from background noise in applications such as radio-frequency demodulation.

Consider the following transfer function for a second order IIR filter with complex-conjugate poles:

$$H_i(z) = \frac{1-r}{(1-re^{j\theta}z^{-1})(1-re^{-j\theta}z^{-1})} = \frac{1-r}{1-2r\cos(\theta)z^{-1}+r^2z^{-2}} \tag{7.10}$$

Figure 7.3 shows the locations of the two poles of this filter. The poles have the form

$$p_1 = re^{j\theta} \qquad p_2 = re^{-j\theta}$$

where $r$ is the distance from the origin, and $\theta$ is the angle of $p_1$ relative to the positive real axis. From the theory of $z$-transforms, we know that a causal filter is stable if and only if its poles are located within the unit circle. This implies that this filter is stable if and only if $|r| < 1$. However, we will see that by locating the poles close to the unit circle, the filter's bandwidth may be made extremely narrow around $\theta$.

This two-pole system is an example of an IIR filter because its impulse response has infinite duration. Any filter with nontrivial poles (not at $z = 0$ or $\pm\infty$) is an IIR filter unless the poles are canceled by zeros.

Calculate the magnitude of the filter's frequency response $\left|H_i(e^{j\omega})\right|$ on $|\omega| < \pi$ for $\theta = \pi/3$ and the following three values of $r$.

i) $r = 0.99$, ii) $r = 0.9$, iii) $r = 0.7$

Put all three plots on the same figure using the subplot command.

<u>INLAB REPORT: Submit the difference equation, system diagram and the analytical expression of the impulse response for $H_i$(z). Also submit the plot of the magnitude of the frequency response for each value of $r$. Explain how the value of $r$ affects this magnitude.</u>

In the following experiment, we will use the filter $H_i(z)$ to separate a modulated sinusoid from background noise. To run the experiment, first download the file **pcm.mat** and load it into the Matlab workspace using the command **load pcm**. Play **pcm** using the **sound** command. Plot 101 samples of the signal for indices (100:200), and then compute the magnitude of the DTFT of 1001 samples of **pcm** using the time indices (100:1100). Plot the magnitude of the DTFT samples versus angular frequency for $|\omega| < \pi$. The two peaks in the spectrum correspond to the center frequency of the modulated signal. The low amplitude wideband content is the background noise. In this exercise, you will use the IIR filter described above to amplify the desired signal, relative to the background noise.

The **pcm** signal is modulated at 3146Hz and sampled at 8kHz. Use these values to calculate the value of $\theta$ for the filter $H_i(z)$. Remember from the sampling theorem that an angular frequency of $2\pi$ corresponds to the sampling frequency.

Write a Matlab function **IIRfilter**(x) that implements the filter $H_i(z)$. In this case, you need to use a **for** loop to implement the recursive difference equation. Use your calculated value of $\theta$ and $r = 0.995$. You can assume that $y[n]$ is equal to 0 for negative values of $n$. Apply the new command **IIRfilter** to the signal **pcm** to separate the desired signal from the background noise, and listen to the filtered signal to hear the effects. Plot the filtered signal for indices (100:200), and then compute the DTFT of 1001 samples of the filtered signal using the time indices (100:1100). Plot the magnitude of this DTFT. In order to see the DTFT around $\omega = \theta$ more clearly, plot also the portion of this DTFT for the values of $\omega$ in the range $[\theta - 0.02, \theta + 0.02]$. Use your calculated value of $\theta$.

<u>**INLAB REPORT:** For both the pcm signal and the filtered output, submit the following:</u>

<u>• The time domain plot of the signal for 101 points.</u>

<u>• The plot of the magnitude of the DTFT computed from 1001 samples of the signal.</u>

<u>• The plot of the magnitude of the DTFT for $\omega$ in the range $[\theta - 0.02, \theta + 0.02]$.</u>

<u>Also hand in the code for the IIRfilter filtering function. Comment on how the signal looks and sounds before and after filtering. How would you expect changes in $r$ to change the filtered output? Would a value of $r = 0.9999999$ be effective for this application? Why might such a value for $r$ be ill-advised? (Consider the spectrum of the desired signal around $\omega = \theta$.)</u>

# 7.5 Lowpass Filter Design Parameters

Download the file **nspeech2.mat** for the following section.

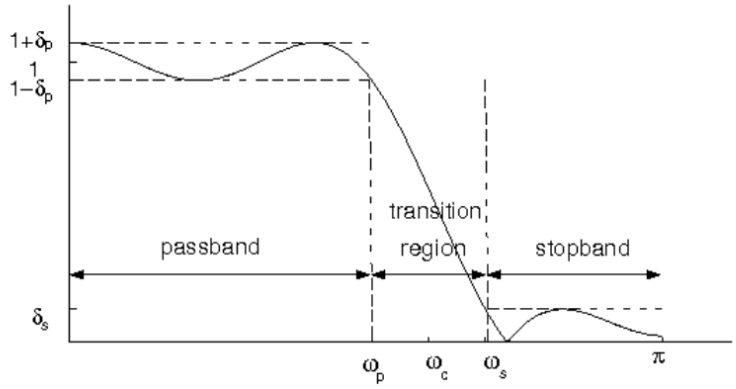


**Figure 7.4**: Tolerance specifications for the frequency response of a filter.

Oftentimes it is necessary to design a good approximation to an ideal lowpass, highpass or bandpass filter. Figure 7.4 illustrates the typical characteristics of a real lowpass filter. The frequencies $|\omega| < \omega_p$ are known as the passband, and the frequencies $\omega_{s<}|\omega| \leq \pi$ are the stopband. For any real lowpass filter, $\omega_p < \omega_s$. The range of frequencies $\omega_p \leq \omega \leq \omega_s$ is known as the transition band. The magnitude of the filter response, $H(e^{j\omega})$, is constrained in the passband and stopband by the following two equations

$$\left|H\left(e^{j\omega}\right) - 1\right| \leq \delta_p \quad \text{for} \quad |\omega| \leq \omega_p$$

$$\left|H\left(e^{j\omega}\right)\right| \leq \delta_s \quad \text{for} \quad \omega_s \leq |\omega| \leq \pi \tag{7.11}$$

where $\delta_p$ and $\delta_s$ are known as the passband and stopband ripple respectively. Most lowpass filter design techniques depend on the specification of these four parameters: $\omega_p$, $\omega_s$, $\delta_p$, and $\delta_s$.

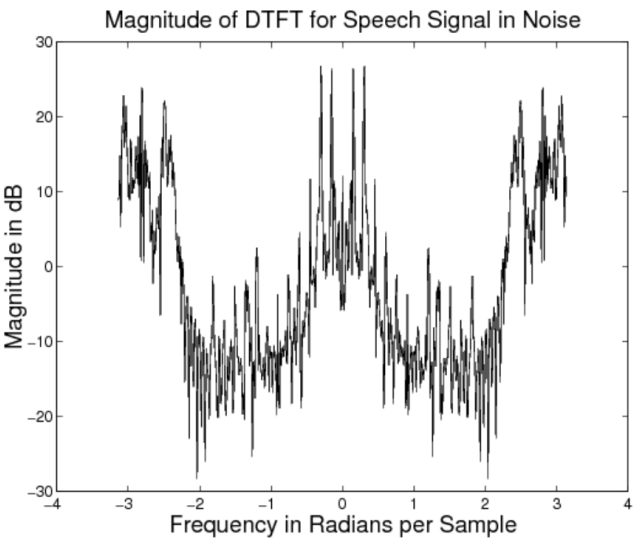


**Figure 7.5**: DTFT of a section of noisy speech.

To illustrate the selection of these parameters consider the problem of filtering out background noise from a speech signal. Figure 7.5 shows the magnitude of the DTFT over a window of such a signal, called **nspeech2**. Notice that there are two main components in **nspeech2**: one at the low frequencies and one at the high. The high frequency signal is noise, and it is band limited to $|\omega| > 2.2$. The low frequency signal is speech and it is band limited to $|\omega| < 1.8$. Download the file **nspeech2.mat** and load it into the Matlab workspace. It contains the signal **nspeech2** from Figure 7.5. Play the **nspeech2** using the **sound** command and note the quality of the speech and background noise.

In the following sections, we will compute lowpass filters for separating the speech and noise using a number of different methods.

# 7.6 Filter Design Using Truncation

Ideally, a lowpass filter with cutoff frequency $\omega_c$ should have a frequency response of

$$H_{ideal}(e^{j\omega}) = \begin{cases} 1 & |\omega| \le \omega_c \\ 0 & \omega_c < |\omega| \le \pi \end{cases} \tag{7.12}$$

and a corresponding impulse response of

$$h_{ideal}[n] = \frac{\omega_c}{\pi}\operatorname{sinc}\left(\frac{\omega_c n}{\pi}\right) \quad \text{for} \quad -\infty < n < \infty \tag{7.13}$$

However, no real filter can have this frequency response because $h_{ideal}(n)$ is infinite in duration.

One method for creating a realizable approximation to an ideal filter is to truncate this impulse response outside of $n \in [-M, M]$.

$$h_{trunc}[n] = \begin{cases} \frac{\omega_c}{\pi}\operatorname{sinc}\left(\frac{\omega_c}{\pi}n\right) & n = -M, \dots, 0, 1, \dots M \\ 0 & \text{otherwise} \end{cases} \tag{7.14}$$

Figure 7.6 shows the magnitude response of the lowpass filter with cutoff frequency $\omega_c = 2.0$, with the impulse response truncated to $n \in [-10, 10]$. Notice the oscillatory behavior of the magnitude response near the cutoff frequency and the large amount of ripple in the stopband.

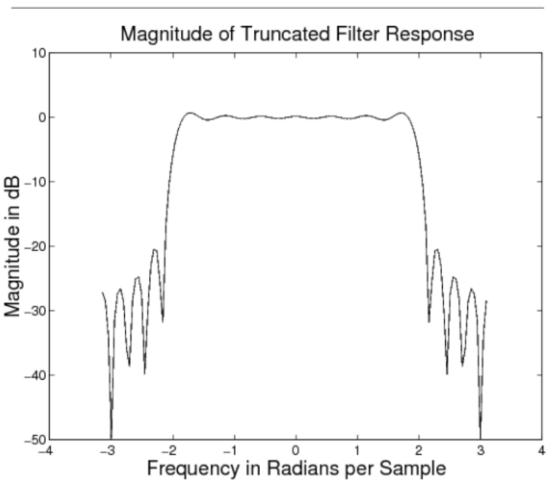


**Figure 7.6:** Frequency response of lowpass filter designed using the truncation method.

Due to the modulation property of the DTFT, the frequency response of the truncated filter is the result of convolving the magnitude response of the ideal filter (a rect) with the DTFT of the truncating window. The DTFT of the truncating window, shown in Figure 7.7, is similar to a sinc function since it is the DTFT of a sampled rectangular window. Notice that this DTFT has very large sidelobes, which lead to large stopband ripple in the final filter.

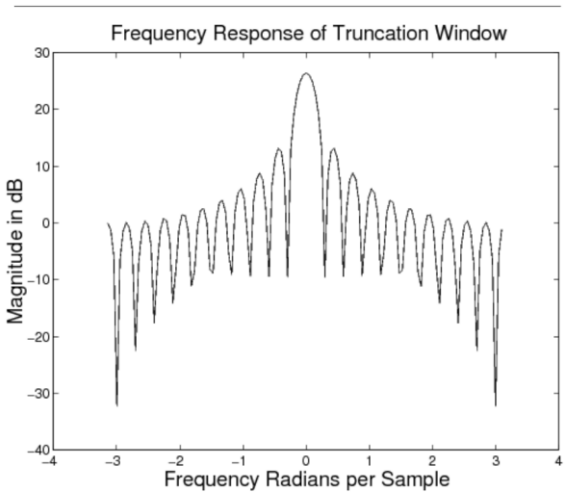


**Figure 7.7**: DTFT of a rectangular window of size 21.

A truncated impulse response is of finite duration, yet the filter is still non causal. In order to make the FIR filter causal, it must be shifted to the right by $M$ units. For a filter of size $N = 2M + 1$ this shifted and truncated filter is given by

$$h[n] = \begin{cases} \frac{\omega_c}{\pi}\text{sinc}\left(\frac{\omega_c}{\pi}\left(n - \frac{N-1}{2}\right)\right) & n = 0,1,\ldots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{7.15}$$

This time shift of $(N-1)/2$ units to the right corresponds to multiplying the frequency response by $e^{-j\omega(N-1)/2}$. It does not affect the magnitude response of the filter, but adds a factor of $-j\omega(N-1)/2$ to the phase response. Such a filter is called linear phase because the phase is a linear function of $\omega$.

It is interesting to see that the filter formula of (7.15) is valid for $N$ both even and odd. While both of these filters are linear phase, they have different characteristics in the time domain. When $N$ is odd, then the value at $n = (N-1)/2$ is sampled at the peak of the sinc function, but when $N$ is even, then the two values at $n = N/2$ and $n = (N/2) - 1$ straddle the peak.

To examine the effect of filter size on the frequency characteristics of the filter, write a Matlab function **LPFtrunc**(N) that computes the truncated and shifted impulse response of size $N$ for a lowpass filter with a cutoff frequency of $\omega_c = 2.0$. For each of the following filter sizes, plot the magnitude of the filter's DTFT in decibels.

**HINTS:** The magnitude of the response in decibels is given by $\left|H_{dB}(e^{j\omega})\right| = 20\log_{10}\left|H(e^{j\omega})\right|$.

Note that the **log** command in Matlab computes the natural logarithm. Therefore, use the **log10** command to compute decibels. To get an accurate representation of the DTFT make sure that you compute at least 512 sample points using the command [X ,w]=DTFT(filter_response,512) .

i) $N = 21$, and ii) $N = 101$

(7.16)

Now download the noisy speech signal **nspeech2.mat**, and load it into the Matlab workspace. Apply the two filters with the sizes specified in (7.16) to this signal. Since these are FIR filters, you can simply convolve them with the audio signal. Listen carefully to the unfiltered and filtered signals, and note the result. Can you hear a difference between the two filtered signals? In order to hear the filtered signals better, you may want to multiply each of them by 2 or 3 before using sound.

**INLAB REPORT**

- Submit the plots of the magnitude response for the two filters (not in decibels). On each of the plots, mark the passband, the transition band and the stopband.
- Submit the plots of the magnitude response in decibels for the two filters.
- Explain how the filter size effects the stopband ripple. Why does it have this effect?
- Comment on the quality of the filtered signals. Does the filter size have a noticeable effect on the audio quality?

# 7.7 Filter Design Using Standard Windows

Download **DTFT.m** for the following section.

We can generalize the idea of truncation by using different windowing functions to truncate an ideal filter's impulse response. Note that by simply truncating the ideal filter's impulse response, we are actually multiplying (or "windowing") the impulse response by a shifted $rect(\,)$ function. This particular type of window is called a **rectangular** window. In general, the impulse response $h[n]$ of the designed filter is related to the impulse response $h_{ideal}[n]$ of the ideal filter by the relation

$$h[n] = w[n]h_{ideal}[n] \tag{7.17}$$

where $w[n]$ is an $N$-point window. We assume that

$$h_{ideal}[n] = \frac{\omega_c}{\pi}\operatorname{sinc}\left(\frac{\omega_c}{\pi}\left(n - \frac{N-1}{2}\right)\right) \tag{7.18}$$

where $\omega_c$ is the cutoff frequency and $N$ is the desired window length.

The rectangular window is defined as

$$w[n] = \begin{cases} 1 & n = 0,1,\ldots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{7.19}$$

As we have seen that the DTFT of $w[n]$ for $N = 21$ is shown in Figure 7.7. The rectangular window is usually not preferred because it leads to the large stopband and passband ripple as shown in Figure 7.6.

More desirable frequency characteristics can be obtained by making a better selection for the window, $w[n]$. In fact, a variety of raised cosine windows are widely used for this purpose. Some popular windows are listed below.

1. Hanning window (as defined in Matlab, command hann(N)):

$$w[n] = \begin{cases} 0.5 - 0.5\cos\frac{2\pi n}{N-1} & n = 0,1,\dots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{7.20}$$

2. Hamming window

$$\omega[n] = \begin{cases} 0.54 - 0.46\cos\frac{2\pi n}{N-1} & n = 0,1,\dots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{7.21}$$

3. Blackman window

$$\omega[n] = \begin{cases} 0.42 - 0.5\cos\frac{2\pi n}{N-1} + 0.08\cos\frac{4\pi n}{N-1} & n = 0,1,\dots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{7.22}$$

In filter design using different truncation windows, there are two key frequency domain effects that are important to the design: the transition band **roll-off**, and the passband and stopband **ripple** (see Figure 7.4). There are two corresponding parameters in the spectrum of each type of window that influence these filter parameters. The filter's roll-off is related to the **width** of center lobe of the window's magnitude spectrum. The **ripple** is influenced by the ratio of the mainlobe amplitude to the first sidelobe amplitude (or difference if using a dB scale). These two window spectrum parameters are **not** independent, and you should see a trend as you examine the spectra for different windows. The theoretical values for the **mainlobe width** and the **peak-to-sidelobe amplitude** are shown in Table 7.1.

**Table 7.1**: Approximate spectral parameters of truncation windows. See reference [1].

| Window (length N) | Mainlobe width | Peak-to-sidelobe amplitude (dB) |
|---|---|---|
| *Rectangular* | $4\pi/N$ | $-13dB$ |
| *Hanning* | $8\pi/N$ | $-32dB$ |
| *Hamming* | $8\pi/N$ | $-43dB$ |
| *Blackman* | $12\pi/N$ | $-58dB$ |

Plot the rectangular, Hamming, Hanning, and Blackman window functions of length 21 on a single figure using the subplot command. You may use the Matlab commands **hamming**, **hann**, and **blackman**. Then compute and plot the DTFT magnitude of each of the four windows. Plot the magnitudes on a decibel scale (i.e., plot $20\log_{10}\left|W(e^{j\omega})\right|$ ). Download and use the function **DTFT.m** to compute the DTFT.

NOTE: Use at least 512 sample points in computing the DTFT by typing the command DTFT(window,512).

Measure the null-to-null mainlobe width (in rad/sample) and the peak-to-sidelobe amplitude (in dB) from the logarithmic magnitude response plot for each window type. The Matlab command **zoom** is helpful for this. Make a table with these values and the theoretical ones.

Now use a Hamming window to design a lowpass filter $h[n]$ with a cutoff frequency of $\omega_c = 2.0$ and length 21. Note: You need to use (7.17) and (7.18) for this design. In the same figure, use subplots to plot the filter's impulse response, and the magnitude of the filter's DTFT in decibels.

**INLAB REPORT:**

1. Submit the figure containing the time domain plots of the four windows.
2. Submit the figure containing the DTFT (in decibels) of the four windows.
3. Submit the table of the measured and theoretical window spectrum parameters. Comment on how close the experimental results matched the ideal values. Also comment on the relation between the width of the mainlobe and the peak-to-sidelobe amplitude.
4. Submit the plots of your designed filter's impulse response and the magnitude of the filter's DTFT.

# 7.8 Filter Design Using the Kaiser Window

Download **nspeech2.mat** for the following section.

The standard windows of the "Filter Design Using Standard Windows" section are an improvement over simple truncation, but these windows still do not allow for arbitrary choices of transition bandwidth and ripple. In 1964, James Kaiser derived a family of near-optimal windows that can be used to design filters which meet or exceed any filter specification. The Kaiser window depends on two parameters: the window length $N$, and a parameter $\beta$ which controls the shape of the window. Large values of $\beta$ reduce the window side lobes and therefore result in reduced passband and stopband ripple. The only restriction in the Kaiser filter design method is that the passband and stopband ripple must be equal in magnitude. Therefore, the Kaiser filter must be designed to meet the smaller of the two ripple constraints:

$$\delta = \min\{\delta_p, \delta_s\} \tag{7.23}$$

The Kaiser window function of length $N$ is given by

$$\omega[n] = \begin{cases} \dfrac{I_0\left(\beta\sqrt{\dfrac{n(N-1-n)}{N-1}}\right)}{I_0(\beta)} & n = 0,1,\ldots,N-1 \\ 0 & \text{otherwise} \end{cases} \tag{7.24}$$

Where $I_0(\cdot)$ is the zero-th order modified Bessel function of the first kind, $N$ is the length of the window, and $\beta$ is the shape parameter.

Kaiser found that values of $\beta$ and $N$ could be chosen to meet any set of design parameters, $(\delta, \omega_P, \omega_S)$, by defining $A = -20\log_{10}\delta$ and using the following two equations:

$$\beta = \begin{cases} 0.1102(A-8.7) & A > 50 \\ \{0.5842(A-21)^{0.4} + 0.07886(A-21) & 21 \le A \le 50 \\ 0.0 & A < 21 \end{cases} \tag{7.25}$$

$$N = \left\lceil 1 + \frac{A-8}{2.285(\omega_s - \omega_p)} \right\rceil \tag{7.26}$$

where $\lceil \cdot \rceil$ is the **ceiling** function, i.e. $\lceil x \rceil$ is the smallest integer which is greater than or equal to $x$.

To further investigate the Kaiser window, plot the Kaiser windows and their DTFT magnitudes (in dB) for $N = 21$ and the following values of $\beta$:

- $\beta = 0$
- $\beta = 1$
- $\beta = 5$

For each case use at least 512 points in the plot of the DTFT.

**HINT:** To create the Kaiser windows, use the Matlab command kaiser(N,beta) command where N is the length of the filter and beta is the shape parameter $\beta$. To insure at least 512 points in the plot use the command DTFT(window,512) when computing the DTFT.

**INLAB REPORT:** Submit the plots of the 3 Kaiser windows and the magnitude of their DTFT's in decibels. Comment on how the value $\beta$ affects the shape of the window and the sidelobes of the DTFT.

Next use a Kaiser window to design a lowpass filter, $h(n)$, using equations (7.17) and (7.18), to remove the noise from the signal in **nspeech2.mat**. To do this, use equations (7.25) and (7.26) to compute the values of $\beta$ and $N$ that will yield the following design specifications:

$$\omega_p = 1.8,\ \omega_c = 2.0,\ \omega_s = 2.2,\ \delta_p = 0.05,\ \delta_s = 0.005$$

The lowpass filter designed with the Kaiser method will automatically have a cut-off frequency centered between $\omega_p$ and $\omega_s$.

$$\omega_c = \frac{\omega_p + \omega_s}{2}$$

Plot the magnitude of the DTFT of $h[n]$ for $|\omega| < \pi$. Create three plots in the same figure: one that shows the entire frequency response, and ones that zoom in on the passband and stopband ripple, respectively. Mark $\omega_p$, $\omega_s$, $\delta_p$, and $\delta_s$ on these plots where appropriate. Note: Since the ripple is measured on a magnitude scale, DO NOT use a decibel scale on this set of plots.

From the Matlab prompt, compute the stopband and passband ripple (do not do this graphically). Record the stopband and passband ripple to three decimal places.

**HINT:** To compute the passband ripple, find the value of the DTFT at frequencies corresponding to the passband using the command H(abs(w)<=1.8) where H is the DTFT of $h[n]$ and w is the corresponding vector of frequencies. Then use this vector to compute the passband ripple. Use a similar procedure for the stopband ripple.

Filter the noisy speech signal in **nspeech2.mat** using the filter you have designed. Then compute the DTFT of 400 samples of the filtered signal starting at time $n = 20000$ (i.e. 20001:20400 ). Plot the magnitude of the DTFT samples in decibels versus frequency in radians for $|\omega| < \pi$. Compare this with the spectrum of the noisy speech signal shown in Figure 7.5. Play the noisy and filtered speech signals back using **sound** and listen to them carefully.

**INLAB REPORT:** Do the following:

1. Submit the values of $\beta$ and $N$ that you computed.
2. Submit the three plots of the filter's magnitude response. Make sure the plots are labeled.
3. Submit the values of the passband and stopband ripple. Does this filter meet the design specifications?
4. Submit the magnitude plot of the DTFT in dB for the filtered signal. Compare this plot to the plot of Figure 7.5.
5. Comment on how the frequency content and the audio quality of the filtered signal have changed after filtering.